

IMPLEMENTATION ASSESSMENT

Galecto Project Audit Implementation, Gaps, Risks, and Enterprise Roadmap

A detailed technical assessment of the current Galecto codebase, comparing the implementation claims in the project documentation with the actual service, data, and frontend wiring present in the repository as of May 10, 2026.

PROJECT Galecto	ASSESSMENT DATE May 10, 2026	SCOPE Docs vs Codebase	OVERALL STATUS Prototype / MVP+
---------------------------------------	--	--	---

Sources reviewed: docs/, apps/, packages/, and service route/controller wiring in the current workspace.

1. Executive Summary

The Galecto project has a strong product direction and a credible microservice-based observability platform shape, but it is not yet implemented to the level claimed in the documentation. The codebase already contains the core building blocks for authentication, telemetry ingestion, log search, trace visualization, replay, alerting, monitoring, project management, and retention management. However, many of these capabilities are only partially wired together, and several high-value enterprise features remain either non-functional, UI-only, or structurally incomplete.

The current state is best described as a promising product prototype or an MVP-plus platform rather than a finished enterprise-grade observability suite.

FEATURE COVERAGE

6.5/10

Most major modules exist, but several are only partially operational.

ARCHITECTURE INTEGRITY

5.5/10

Good service decomposition, but route, auth, and schema gaps weaken the platform.

ENTERPRISE READINESS

3.5/10

Missing durability, security hardening, governance, and operational maturity.

DOCUMENTATION ACCURACY

4/10

Docs overstate completion relative to current implementation.

High-Level Conclusion

The platform direction is solid. The biggest immediate need is not adding flashy new UI, but closing the core correctness gaps: route prefix mismatches, auth model inconsistency, missing trace parent linkage, incomplete project APIs, hardcoded retention behavior, and non-persistent alerting.

Important: The file docs/CURRENT_IMPLEMENTATION.md currently states that all phases are complete. The codebase does not support that claim today. This mismatch should be corrected to avoid confusion for developers, reviewers, stakeholders, and potential customers.

2. Assessment Method

This report was prepared by comparing the implementation claims in the documentation against the actual source code present in the repository. The review focused on the following areas:

- Backend service startup and route wiring
- Authentication and authorization flow
- Data model and persistence capabilities
- Telemetry ingestion, trace stitching, log search, replay, alerting, and monitoring
- Frontend route coverage and whether UI actions are backed by working APIs
- Operational readiness and enterprise-grade concerns

Key Source Files Reviewed

AREA	REPRESENTATIVE FILES	PURPOSE
Implementation claims	docs/CURRENT_IMPLEMENTATION.md docs/GALECTO_TECHNICAL_GUIDE.md docs/GALECTO_DEEP_DIVE.md	Stated feature scope and maturity level
API gateway	apps/api-gateway/src/app.ts apps/api-gateway/src/routes/index.ts apps/api-gateway/src/middleware/auth.ts apps/api-gateway/src/controllers/replay.controller.ts	Ingress, replay, auth, event production
Query service	apps/query-service/src/server.ts apps/query-service/src/routes/trace.routes.ts apps/query-service/src/controllers/trace.controller.ts apps/query-service/src/controllers/log.controller.ts	Trace, anomaly, metric, and log query behavior
Auth and projects	apps/auth-service/src/app.ts apps/auth-service/src/controllers/auth.controller.ts apps/auth-service/src/controllers/project.controller.ts apps/auth-service/prisma/schema.prisma	User, org, project, and API key management
Telemetry persistence	apps/log-service/src/server.ts apps/log-service/src/workers/retention.worker.ts packages/clickhouse/src/client.ts	Event storage, schema, and retention
Alerting	apps/alert-service/src/server.ts	Real-time anomaly detection and incident handling

AREA	REPRESENTATIVE FILES	PURPOSE
Frontend	apps/frontend/src/app/* apps/frontend/src/lib/apiClient.ts apps/frontend/src/context/AuthContext.tsx	Feature exposure and end-to-end flow checks

3. Feature-by-Feature Implementation Status

FEATURE AREA	STATUS	ASSESSMENT	EVIDENCE
SaaS onboarding and auth bootstrap	PARTIAL	User signup creates organization, default project, and initial API key. Login and token issuance exist. However, downstream authenticated project handling is incomplete.	<code>apps/auth-service/src/services/auth.service.ts</code> <code>apps/auth-service/src/controllers/auth.controller.ts</code>
Frontend auth session handling	IMPLEMENTED	Browser session persistence and route redirection are present through local storage and the auth context.	<code>apps/frontend/src/context/AuthContext.tsx</code>
Telemetry ingestion through gateway	PARTIAL	Gateway emits events to Kafka, but the auth model for ingest depends on Redis-cached API keys and does not align cleanly with JWT-based frontend usage.	<code>apps/api-gateway/src/routes/index.ts</code> <code>apps/api-gateway/src/middleware/auth.ts</code>
Kafka to ClickHouse persistence	IMPLEMENTED	Log service consumes Kafka events and inserts them into ClickHouse.	<code>apps/log-service/src/server.ts</code>

FEATURE AREA	STATUS	ASSESSMENT	EVIDENCE
Trace list and trace detail API	PARTIAL	Controller logic exists, but the service route registration is mismatched with frontend expectations.	apps/query-service/src/server.ts apps/query-service/src/routes/trace.routes.ts
Hierarchical trace stitching	PARTIAL	Tree-building logic exists, but the stored event schema does not preserve parent span relationships, which limits correctness.	apps/query-service/src/controllers/trace.controller.ts packages/clickhouse/src/client.ts
Logs explorer	IMPLEMENTED	Search and service filters are supported through ClickHouse-backed query APIs.	apps/query-service/src/controllers/log.controller.ts apps/frontend/src/app/logs/page.tsx
Replay controller	PARTIAL	Replay logic exists, but it depends on request metadata being stored in a very specific payload shape, and the UI auth path is not aligned with gateway auth expectations.	apps/api-gateway/src/controllers/replay.controller.ts apps/frontend/src/app/replay/page.tsx

FEATURE AREA	STATUS	ASSESSMENT	EVIDENCE
Alert detection	PARTIAL	Latency and error alerts are generated from Kafka events, but the storage is in-memory and not suitable for production.	<code>apps/alert-service/src/server.ts</code>
Monitoring / metrics	PARTIAL	Metrics query logic exists, but frontend wiring depends on routes that appear mismatched, and some dashboard values remain static placeholders.	<code>apps/query-service/src/controllers/trace.controller.ts</code> <code>apps/frontend/src/app/monitoring/page.tsx</code> <code>apps/frontend/src/app/dashboard/page.tsx</code>
Project management	PARTIAL	Project and API key data models exist, but project routes assume authenticated user context that is not fully wired.	<code>apps/auth-service/src/controllers/project.controller.ts</code> <code>apps/auth-service/src/routes/project.routes.ts</code>
Retention management	PARTIAL	A scheduled retention worker exists, but retention is hardcoded to 30 days and not connected to org settings or the frontend update control.	<code>apps/log-service/src/workers/retention.worker.ts</code> <code>apps/auth-service/prisma/schema.prisma</code> <code>apps/frontend/src/app/settings/page.tsx</code>

FEATURE AREA	STATUS	ASSESSMENT	EVIDENCE
Developer hub	PARTIAL	The documentation page is present, but the referenced SDK package is not implemented in the repository.	<code>apps/frontend/src/app/developer/page.tsx</code>
Enterprise-grade reliability and governance	MISSING	Core production controls such as durable alerting, RBAC, auditability, policy enforcement, rate limiting, encryption strategy, and observability of the platform itself are not yet present.	Cross-cutting gap across services

4. What Is Implemented Well

Service Decomposition

The project is already organized into focused services for gateway, auth, query, log processing, alerting, and frontend presentation. That separation will help future scaling and team ownership.

Clear Product Direction

The feature set is cohesive. Traces, logs, metrics, replay, alerts, and project-level control all fit together into a strong observability narrative.

ClickHouse-Centered Querying

The use of ClickHouse for telemetry analytics is an appropriate choice for observability-style workloads and supports low-latency aggregation patterns.

Frontend Surface Area

The application already has a broad UI footprint, which means the product vision is tangible and not just backend scaffolding.

Concrete Strengths

- Signup flow creates organization, default project, and initial API key at bootstrap time.
- Kafka-based decoupling between ingestion and analytics is already present.
- The query layer already contains logic for traces, anomalies, metrics, and logs.
- The alert-service already has a working consumer loop for basic anomaly heuristics.
- The frontend has dedicated workspaces for traces, logs, replay, alerts, monitoring, developer setup, and settings.

5. What Is Going Wrong in the Project

Summary: the main problem is not lack of ambition. The main problem is inconsistency between the documented architecture, the UI assumptions, the auth model, and the actual backend route and data wiring.

5.1 Route Prefix Mismatches Break End-to-End Flows

The frontend calls trace APIs using paths such as `/api/v1/traces`, `/api/v1/traces/anomalies`, and `/api/v1/traces/metrics`. However, the query service registers trace routes at the root level rather than under the `/api/v1/traces` prefix.

Evidence: `apps/query-service/src/server.ts` registers `traceRoutes` directly, and `apps/query-service/src/routes/trace.routes.ts` defines paths as `/`, `/anomalies`, `/metrics`, and `/:traceId`.

5.2 Auth Model Is Split Between JWT Sessions and API Key Validation

The frontend authenticates with JWTs stored in local storage, while the gateway ingest and replay endpoints expect Bearer tokens to be API keys cached in Redis. This means the UI and gateway are speaking two different auth languages.

Evidence: `apps/frontend/src/lib/apiClient.ts` sends the browser token as a Bearer token, while `apps/api-gateway/src/middleware/auth.ts` looks that same Bearer token up as `apikey:{token}` in Redis.

5.3 Project APIs Assume User Context That Is Not Established

Project controllers read `(req as any).user.organizationId`, but there is no visible middleware on those routes populating `req.user` through JWT verification. As a result, the project management path is structurally incomplete.

5.4 Trace Schema Cannot Fully Support Claimed Hierarchical Stitching

The trace controller builds trees using `parent_span_id`, but the ClickHouse events table does not store `parent_span_id` at all. This limits accuracy and makes some of the causality claims stronger than the actual stored data model.

5.5 Replay Depends on Narrow Metadata Assumptions

Replay logic reads a single event payload, parses method, URL, headers, and body from it, and replays through the gateway. That can work for controlled demos, but it is fragile for production use without a dedicated replay-safe request capture schema and redaction strategy.

5.6 Alerts Are Not Durable

Incidents are held in memory only. If the alert-service restarts, the alert history is lost. This makes the current incident center unsuitable for enterprise operations, auditing, or shift handoffs.

5.7 Several UI Controls Are Presentational Only

Buttons such as + New Key, Rotate Keys, Create New Rule, View Replay History, and the retention update control do not appear to be connected to complete backend capabilities.

6. Detailed Gap Analysis by Domain

6.1 Authentication and Tenant Isolation

- JWT issuance exists, but JWT verification and role-based request authorization are not consistently enforced across services.
- Gateway API key auth relies on Redis cache presence rather than a robust lookup and rotation strategy.
- The platform does not yet clearly separate human user auth from machine ingest auth in a clean way.
- There is no visible RBAC model beyond a basic Role enum.

6.2 Data Model and Persistence

- ClickHouse event schema is too narrow for a serious tracing/replay product. It lacks explicit parent span storage, status normalization, request metadata columns, replay metadata, and lifecycle fields.
- Alerts are not persisted.
- Retention settings are modeled in Postgres but not operationalized in the worker logic.
- The seed path appears inconsistent with the current Prisma schema expectations, which suggests schema and seed drift.

6.3 Observability Product Depth

- Metrics are currently limited to simple aggregates derived from generic event payloads.
- There is no evidence of SLOs, burn-rate alerts, service dependency maps, or release/environment correlation.
- Replay lacks versioning, replay comparison summaries, request redaction, and outcome storage.

6.4 Frontend-to-Backend Contract Quality

- Several pages assume APIs that are not actually exposed under matching route prefixes.
- Some dashboard visualizations still contain static or decorative data rather than live domain-backed metrics.
- There is no visible contract layer or shared API schema enforcement between frontend and backend.

6.5 Operational Readiness

- No visible test coverage for critical flows.
- No visible deployment hardening, health dependency checks, circuit breaking, or backpressure strategies.
- No durable audit trail for admin actions, alert resolutions, key rotations, or replay usage.
- No evidence of compliance-focused controls such as secret management, encryption policy, or data retention governance per tenant.

7. Enterprise-Level Improvements Recommended

7.1 Foundation Fixes Required Before Scaling

PRIORITY	RECOMMENDATION	WHY IT MATTERS
P0	Unify route prefixes and frontend API contracts	Restores basic feature functionality and reduces false UI confidence.
P0	Separate user JWT auth from telemetry API key auth	Clarifies security boundaries and prevents broken cross-service assumptions.
P0	Store parent span IDs and structured request/response metadata	Enables reliable trace trees and robust replay.
P0	Persist alerts and replay executions in Postgres	Makes incidents and replay history durable and auditable.
P1	Wire project, API key, and retention controls end to end	Turns current UI concepts into real tenant management capabilities.

7.2 Product Features Needed for a Real Enterprise Platform

- Multi-environment project structure: dev, staging, prod, and region-aware segmentation.
- Role-based access control with organization admin, developer, observer, and billing/admin roles.
- API key lifecycle features: create, rotate, revoke, expiry, scope, and audit history.
- Persistent incident management with severity, assignment, notes, SLA timestamps, and change history.
- Notification integrations for Slack, email, Teams, PagerDuty, and webhooks.
- Saved searches, dashboards, and user-configured alert rules.
- Release correlation and deploy markers for incident analysis.
- OTLP and OpenTelemetry SDK support instead of only custom event ingestion.
- Replay safeguards: PII masking, target environment controls, allowlists, and redaction policies.
- Service map, dependency map, anomaly trends, and SLO burn-rate views.

7.3 Engineering Maturity Improvements

- Add shared typed API contracts between frontend and services.
- Add integration tests for signup, login, project creation, trace query, replay, and alert resolution.
- Add schema migration discipline for ClickHouse and Postgres.
- Add structured error taxonomies and consistent response envelopes.
- Add internal telemetry for the Galecto platform itself.
- Add CI pipelines with lint, typecheck, integration test, and migration validation stages.

8. Recommended Roadmap

Phase 1: Stabilize the Core Platform

- Fix query-service route prefixes to match frontend usage.
- Introduce JWT verification middleware for user-facing backend routes.
- Introduce a clear API key verification flow for machine ingest and replay authorization.
- Normalize event schema and add missing tracing columns.
- Make project creation, key generation, and retention settings work end to end.

Phase 2: Make Core Data Durable and Trustworthy

- Persist alerts, replay executions, and replay results.
- Add retention-by-tenant behavior from Postgres configuration.
- Add replay history, audit trails, and secure redaction rules.
- Replace placeholder UI metrics with live calculated values.

Phase 3: Add Enterprise Controls

- RBAC and scoped API keys.
- Incident workflow, acknowledgement, escalation, and notification integrations.
- Saved dashboards, alert rule builder, and alert suppression windows.
- Environment-aware and project-aware governance controls.

Phase 4: Deepen the Product Advantage

- OpenTelemetry compatibility and SDK packaging.
- Trace diffing and release-over-release comparison.
- Root-cause suggestions, blast radius analysis, and anomaly clustering.
- Replay safety policies and enterprise audit exports.

9. Recommended Order of Work

The most effective way to proceed is not feature by feature in random order. This project should be handled in a dependency-aware sequence, because several visible UI issues are actually caused by deeper backend contract and schema problems. The right approach is to stabilize the foundations first, then wire core product flows, and only after that expand into enterprise capabilities.

9.1 What to Look At First

STEP	AREA / SERVICE	WHY IT COMES FIRST	MAIN GOAL
1	Project-wide contracts and route mapping	Before fixing features, the team must know which frontend pages call which backend routes and whether the paths match.	Create a clear map of frontend pages to actual service endpoints.
2	Authentication model across <code>frontend</code> , <code>api-gateway</code> , and <code>auth-service</code>	Auth inconsistency is currently breaking or weakening multiple flows at once.	Separate human-user JWT auth from machine API-key auth and enforce both correctly.
3	<code>query-service</code> route structure	The traces, anomalies, and metrics pages depend on this service and currently appear misaligned.	Make route prefixes and handlers match frontend expectations exactly.
4	Telemetry schema in <code>log-service</code> and <code>packages/clickhouse</code>	Trace quality, replay correctness, and metrics depth all depend on what gets stored.	Add missing fields such as parent span ID and normalized request/response metadata.
5	<code>auth-service</code> projects, API keys, retention settings	These are platform-control capabilities and should work before advanced UI polish.	Make project creation, key generation, and tenant settings functional end to end.
6	Core product flows: traces, logs, replay, alerts, monitoring	Once contracts and storage are correct, these features can be made reliable instead of cosmetic.	Turn each major workspace into a complete working user journey.
7	Enterprise features and governance	These are important, but they should be built on a stable platform core.	Add durability, RBAC, notificationing, auditing, compliance, and operational controls.

9.2 Recommended Sequential Approach

1. Start with a service and route audit. Confirm which frontend pages depend on `api-gateway`, `query-service`, `auth-service`, and `alert-service`. Document the expected endpoint path, the actual registered path, the auth method used, and whether the page is currently functional.
2. Fix authentication and authorization next. Decide the platform contract clearly: browser users should authenticate with JWT sessions, while telemetry producers should use scoped API keys. Then enforce that decision across the gateway and service routes.
3. Repair the `query-service` before improving the dashboards. The traces, anomalies, and metrics pages depend on `query-service` behavior, so the service route prefixes and response shapes must be stabilized before doing more frontend work.
4. Upgrade the event schema and ingestion model. Add the fields necessary for real trace hierarchies, replay-safe capture, environment/project tagging, and stronger metrics. Without this step, later observability features will remain shallow.
5. Complete project and platform settings in `auth-service`. Make sure project listing, project creation, API key generation, API key caching or lookup, and retention policy updates all work in a real end-to-end flow.
6. Then stabilize the five user-facing product pillars in order: traces first, logs second, replay third, alerts fourth, monitoring fifth. This order works well because traces and logs are the base evidence layer, replay depends on both, alerts depend on meaningful telemetry, and monitoring becomes more trustworthy once the underlying data model is solid.
7. Only after the above should the team move into enterprise expansion work such as RBAC, incident lifecycle workflows, notifications, SDK packaging, compliance features, and advanced analytics.

9.3 Best Service-by-Service Execution Order

ORDER	SERVICE / LAYER	WHAT TO DO THERE
1	apps/frontend + service contract mapping	List every frontend page, the API it calls, expected payload shape, expected auth, and whether the current backend supports it.
2	apps/api-gateway	Clarify gateway responsibilities: ingest, replay orchestration, auth forwarding, and possibly proxying user-facing backend calls where appropriate.
3	apps/auth-service	Fix JWT verification, user context injection, project APIs, API key lifecycle behavior, and retention configuration endpoints.
4	apps/query-service	Fix prefixes, response contracts, anomaly and metrics routes, and prepare for stronger trace assembly logic.

ORDER	SERVICE / LAYER	WHAT TO DO THERE
5	apps/log-service + packages/clickhouse + packages/types	Expand the event schema, improve insertion shape, and support retention and richer query semantics.
6	apps/alert-service	Move alerts from in-memory behavior to persisted incidents and add rule management foundations.
7	frontend feature hardening	Once the backend becomes trustworthy, connect the currently placeholder actions and replace static values with live data.

9.4 Suggested Feature Order

- **Feature 1:** Authentication and access model
- **Feature 2:** Trace list and trace detail correctness
- **Feature 3:** Log search stability
- **Feature 4:** Replay capture and replay execution reliability
- **Feature 5:** Project management, API keys, and retention settings
- **Feature 6:** Persistent alerting and incident workflows
- **Feature 7:** Monitoring and dashboard realism
- **Feature 8:** Enterprise governance, RBAC, notifications, auditability, and SDK maturity

Practical advice: do not start by polishing the UI further. The first pass should focus on contracts, auth, routes, schema, and persistence. Otherwise the team risks spending time beautifying pages that are still backed by incomplete or inconsistent backend behavior.

10. Market Competition and Galecto USP

To understand where Galecto fits, it is important to compare it not only against generic monitoring tools, but against modern observability platforms that already combine logs, metrics, traces, real user monitoring, session replay, and incident workflows. As of May 10, 2026, Galecto's closest competition is not a single company but a cluster of platforms serving adjacent needs at different levels of maturity.

10.1 Primary Market Competitors

COMPETITOR	PRIMARY STRENGTH	WHY IT COMPETES WITH GALECTO
Datadog	Enterprise full-stack observability with APM, logs, traces, RUM, and session replay	Competes on unified troubleshooting across frontend and backend with strong enterprise maturity.
New Relic	Unified observability platform with large integration footprint and AI-assisted analysis	Competes on end-to-end telemetry unification, APM, browser monitoring, and intelligent analysis.
Grafana Cloud	Open and composable observability stack with logs, metrics, traces, and frontend observability	Competes on openness, observability breadth, and correlation across telemetry types.
Honeycomb	Deep high-cardinality observability and fast distributed tracing investigation	Competes on rapid root-cause analysis and rich distributed tracing exploration.
SigNoz	Open-source unified observability alternative to Datadog/New Relic	Competes on cost-sensitive full-stack observability using open standards.
HyperDX	Open-source production debugging with session replay, logs, traces, metrics, and errors	Competes most directly with Galecto's current UI direction and ClickHouse-centered stack story.
highlight.io	Open-source developer-friendly monitoring with replay and error-focused debugging	Competes in the lightweight, developer-first observability segment.
Sentry	Error monitoring, tracing, profiling, and replay with strong app/frontend adoption	Competes where debugging starts from user-facing issues, crashes, and replay-linked failures.
VORTEX	Production web traffic inspection and replay	Competes with Galecto's replay idea more than with its broader observability story.

10.2 Where Galecto Fits in This Market

Galecto does not currently compete by breadth, maturity, or enterprise completeness. Today, the larger platforms clearly exceed it in stability, integrations, governance, alerting depth, access control, and operational trust. Galecto's most credible market position is different: it can compete as a more

opinionated platform built specifically around debugging causality and verifying fixes, rather than only visualizing telemetry.

10.3 Galecto's Strongest USP

Galecto's clearest differentiator is this: not just observability, but **causality replay for fix verification**.

Most current observability tools help teams answer questions like:

- What failed?
- Where did latency increase?
- Which service or dependency was involved?
- What did the user experience look like?

Galecto's concept aims to go one step further:

- Capture the request and context behind a failure.
- Reconstruct the causality path.
- Replay the request in a controlled manner.
- Compare old and new outcomes to validate whether a fix actually worked.

That workflow is the most compelling strategic differentiator in the project. It turns observability from a read-only diagnosis tool into a verification-oriented debugging system.

10.4 USP Compared with Current Competitors

COMPARISON AREA	TYPICAL COMPETITORS	POTENTIAL GALECTO ADVANTAGE
Logs, metrics, traces	Very strong and mature across Datadog, New Relic, Grafana, Honeycomb, SigNoz	No near-term advantage. Galecto should not try to win on raw breadth first.
Session replay / user replay	Strong in Datadog, Sentry, HyperDX, highlight.io	Galecto can differentiate if replay is tied to backend request causality rather than only frontend playback.
Distributed tracing	Strong and production-proven in Honeycomb, Datadog, New Relic, Grafana	Galecto can stand out only if trace views directly lead into replay-based validation workflows.
Incident investigation	Mature ecosystem support, alerts, integrations, and collaboration tooling	Galecto can become distinctive by reducing the path from incident to reproduced proof of fix.

COMPARISON AREA	TYPICAL COMPETITORS	POTENTIAL GALECTO ADVANTAGE
Open-source cost story	SigNoz, HyperDX, highlight.io already compete here	Galecto should not rely only on being cheaper; it needs a sharper workflow differentiator.

10.5 Honest Positioning Assessment

At present, Galecto's USP is stronger as a product concept than as a production-ready implementation. That is not a weakness in vision, but it does mean positioning must be honest. The project should not currently be presented as a broader or more complete alternative to Datadog, New Relic, or Grafana Cloud. Instead, it should be positioned as a platform with a promising differentiator in replay-centered debugging and incident verification.

The most accurate near-term positioning would be:

Galecto is best framed as an observability plus production replay verification platform, not just another observability dashboard.

10.6 Strategic Recommendation

- Do not try to beat large incumbents on total feature count in the short term.
- Do not pitch the platform as simply a cheaper Datadog or New Relic alternative.
- Double down on the trace-to-replay-to-verification workflow as the core identity.
- Build credibility first by making replay technically real, safe, and auditable.
- Then expand outward into enterprise-grade observability capabilities around that core loop.

11. Final Verdict

Galecto is not a broken idea. It is an under-finished implementation of a strong idea.

The project already demonstrates meaningful progress in system decomposition, frontend experience, Kafka-to-ClickHouse data movement, and the broad outline of an observability platform. The biggest challenge is that the current codebase has not yet crossed the line from feature-shaped prototype to production-grade enterprise system.

If the team first closes the correctness gaps, aligns the auth and route contracts, expands the data model, and persists key operational entities, the project could mature into a serious platform. Without those changes, the current documentation will continue to overstate the true implementation level.

Bottom line: the project is best represented today as **a strong MVP with enterprise ambition**, not as a fully complete enterprise observability suite.

Immediate Next Actions Recommended

1. Correct documentation to reflect actual status.
2. Fix the route and auth mismatches that currently block core flows.
3. Upgrade the telemetry schema to support real trace and replay correctness.
4. Persist alerts, replay history, and configuration changes.
5. Build test coverage around the most critical user journeys.

Prepared from repository inspection on May 10, 2026.